计算机学院_计算机网络_课程实验报告

实验题目： 实验		学号：202300130185
日期：2025.9.18	班级：23.1	姓名：曲东芸
Email：1718398913@qq.com		

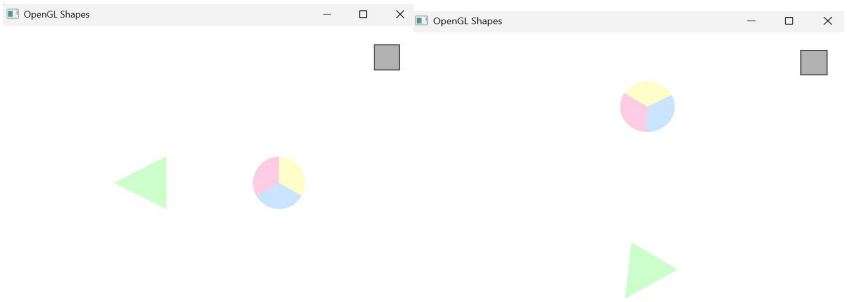
实验内容：

实验 0：OpenGL 基本使用

实验目的：自行配置 OpenGL 环境，熟悉使用 OpenGL

基本要求：在屏幕上绘制出一个三角形和一个圆形，分别填充上你喜欢的颜色，可以使用鼠标分别拖动和旋转他们

运行结果：鼠标拖动可以移动三角形和圆形，按住 shift 并用鼠标拖动可以让其旋转。



实验内容：

创建一个窗口显示三角形和圆形，支持两种交互：直接拖动图形和按住 shift 旋转图形

1. **数据结构 shape**:用一个结构体统一管理图形的所有属性（形状、位置、状态等），便于批量处理（如绘制、交互)

```
// 图形数据结构
struct Shape {
    enum Type { TRIANGLE, CIRCLE } type;//图形类型
    std::vector<float> vertices;        //顶点数据
    unsigned int VAO, VBO;              //OPENGL缓冲区对象，用于存储
    glm::vec3 position;                 //图形在屏幕上的位置
    float rotation;     //旋转角度
    float scale;        //缩放比例
    bool isDraggingPosition;  // 拖动位置
    bool isDraggingRotation;  // 拖动旋转
    glm::vec2 dragStart;      // 用于位置拖动的起始偏移
    glm::vec2 rotationStart;  // 用于旋转拖动的起始位置
};
```

2. **初始化**：搭建环境，初始化 GLFW 窗口系统，初始化 GLAD 用以加载 OPENGL 函数，设置窗口回调，来处理窗口大小变化、鼠标输入

```
// GLFW初始化
glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE)

// 创建窗口
GLFWwindow* window = glfwCreateWindow(SCR_WIDTH, SCR_HEIGHT,
if (window == NULL)
{//检查窗口是否创建成功
    std::cout << "Failed to create GLFW window" << std::endl
    glfwTerminate();
    return -1;
}
//将窗口设置为当前上下文：之后的opengl命令都会作用在这个窗口
glfwMakeContextCurrent(window);
//设置窗口大小变化时的回调函数：保证窗口缩放时渲染内容能正确适配新
glfwSetFramebufferSizeCallback(window, framebuffer_size_callb
//设置鼠标按钮点击的回调函数：点击鼠标时回调函数被触发，实现鼠标点
glfwSetMouseButtonCallback(window, mouse_button_callback);
//设置鼠标移动时的回调函数
glfwSetCursorPosCallback(window, cursor_position_callback);
```

```
// 初始化GLAD，负责加载具体的OPENGL函数
if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress))
{
    std::cout << "Failed to initialize GLAD" << std::endl;
    return -1;
}

// 启用混合模式，使图形边缘更平滑
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

// 创建着色器程序
unsigned int shaderProgram = createShaderProgram();
if (!shaderProgram) return -1;

// 设置投影矩阵
glm::mat4 projection = glm::ortho(0.0f, (float)SCR_WIDTH, 0.0
glUseProgram(shaderProgram);//激活着色器
glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "proje
```

3. **着色器程序**：着色器是运行在显卡上的小程序，负责将顶点数据转换为屏幕像素。代码中包含两个着色器：顶点着色器：处理顶点位置，通过矩阵运算将图形 “放置” 到屏幕指定位置；片段着色器：处理每个像素的颜色，根据图形类型（三角形 / 圆形）上色

```
const char *vertexShaderSource = "#version 330 core\n"
    "layout (location = 0) in vec2 aPos;\n"
    "out vec2 fragPos;\n"
    "uniform mat4 model;\n"
    "uniform mat4 projection;\n"
    "uniform int isButton;\n"
    "void main()\n"
    "{\n"
    "   gl_Position = projection * model * vec4(aPos, 0.0, 1.0)
    "   fragPos = aPos;\n"
    "}\0";
//片段着色器负责计算每个像素的颜色
const char *fragmentShaderSource = "#version 330 core\n"
    "in vec2 fragPos;\n"
```

4. **图形数据准备**：为每个图形定义顶点数据，VAO：顶点数组对象，记录顶点数据的格式（如每个顶点有 2 个坐标值）。VBO：顶点缓冲对象，将顶点数据从内存复制到显卡内存（绘制时直接从显卡读取，速度更快）。

   圆形创建通过“三角形扇”模拟，用大量小三角形逼近圆形，segments 越大，圆形越平滑

```
// 创建三角形
void createTriangle(Shape& shape)
{
    // 三角形顶点（居中在原点）
    float vertices[] = {
        0.0f,  50.0f,  // 上顶点
       -50.0f, -50.0f,  // 左下
        50.0f, -50.0f   // 右下
    };

    // 复制顶点数据
    shape.vertices.assign(vertices, vertices + sizeof(vertices) /

    // 创建VAO和VBO
    glGenVertexArrays(1, &shape.VAO);
    glGenBuffers(1, &shape.VBO);

    glBindVertexArray(shape.VAO);
    glBindBuffer(GL_ARRAY_BUFFER, shape.VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_S

    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(flo
    glEnableVertexAttribArray(0);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
```

5. **矩阵变换**，控制图形位置，旋转，缩放

```
// 创建模型矩阵:控制图形的位置旋转缩放
glm::mat4 model = glm::mat4(1.0f);
model = glm::translate(model, shape.position);
model = glm::rotate(model, glm::radians(shape.rotati
model = glm::scale(model, glm::vec3(shape.scale, sha
```

6. **渲染循环**：绘制所有图形

```
// 渲染循环
while (!glfwWindowShouldClose(window))
{
    // 计算帧时间
    float currentFrame = glfwGetTime();
    float deltaTime = currentFrame - lastFrameTime;
    lastFrameTime = currentFrame;

    // 清屏 - 白色背景
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // 使用着色器程序
    glUseProgram(shaderProgram);

    // 绘制所有图形
    for (size_t i = 0; i < shapes.size(); i++)
    {
        const Shape& shape = shapes[i];

        // 设置uniform变量
        glUniform1i(isCircleLoc, shape.type == Shape::CIR

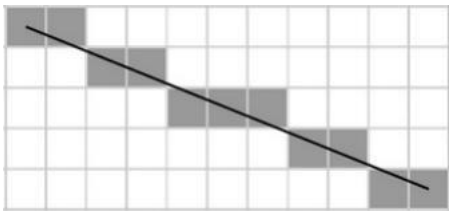
        // 创建模型矩阵:控制图形的位置旋转缩放
```

7. **鼠标交互**：通过处理鼠标点击和鼠标移动两个函数实现交互

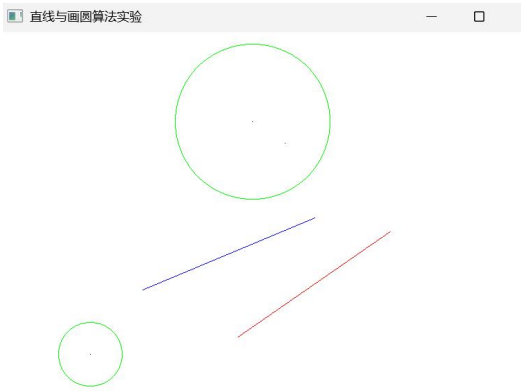
实验 1：直线与画圆算法

实验目的：理解光栅化，掌握几何数据如何经过一系列变换后转化为像素从而呈现在显示设备上。

基本要求：实现 DDA 和 Bresenham 画线算法；实现画圆算法，请勿直接调用 OpenGL 库中提供的绘制线段和圆的函数，需手动模拟如何通过绘制像素来显示几何图形，效果可参考下图。（界面显示的是“虚拟的像素”）；可通过交互操作来确定线段的起点和终点。



运行结果：



实验内容：通过鼠标点击确定直线的起点 / 终点或圆的圆心 / 半径，然后用三种算法（DDA 直线、Bresenham 直线、中点画圆）手动计算需要点亮的像素，最终在屏幕上渲染这些 “虚拟像素”。不调用 OpenGL 的绘制函数，而是自己计算每个像素的位置，模拟光栅化过程。

1.数据结构与变量

struct Pixel：存储单个像素的信息，包括坐标（pos）和颜色（color）。

std::vector<Pixel> pixels：保存所有需要绘制的像素，是 “手动光栅化” 的核心数据容器。

交互相关变量：

linePoints：存储直线的起点和终点（最多 2 个点）。

circleCenter 和 hasCircleCenter：记录圆的圆心及是否已选择圆心。

drawingMode：当前绘制模式（1=DDA 直线，2=Bresenham 直线，3 = 圆）。

```cpp
// 存储绘制的像素点信息
struct Pixel {
    glm::vec2 pos;//像素的2D坐标
    glm::vec3 color;//像素的颜色
};

std::vector<Pixel> pixels;          // 所有绘制的像素
std::vector<glm::vec2> linePoints;  // 线段的点
glm::vec2 circleCenter;             // 圆心
bool hasCircleCenter = false;       // 是否已选择圆心
int drawingMode = 0;                // 0: 未选择, 1:
```

2.OpenGL 初始化与渲染准备

窗口与上下文设置：

用 GLFW 创建 800x600 的窗口，设置 OpenGL 3.3 核心模式。

绑定窗口上下文，初始化 GLAD（加载 OpenGL 函数）。

着色器程序：

顶点着色器：接收像素坐标（aPos），通过正交投影矩阵（projection）转换为屏幕坐标。

片段着色器：接收颜色（color），为像素设置最终颜色。

作用：将我们手动计算的像素坐标和颜色传递给 GPU，最终渲染到屏幕上。

正交投影矩阵：

通过 glm::ortho(0.0f, SCR_WIDTH, 0.0f, SCR_HEIGHT, ...)设置，确保像素坐标（如(x,y)）直接对应屏幕像素位置（左上角为(0,0)，右下角为(800,600)），避免透视变换干扰。

```cpp
// GLFW初始化
glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

// 创建窗口
GLFWwindow* window = glfwCreateWindow(SCR_WIDTH, SCR_HEIGHT, "
if (window == NULL)
{
    std::cout << "Failed to create GLFW window" << std::endl;
    glfwTerminate();
    return -1;
}
glfwMakeContextCurrent(window);
glfwSetFramebufferSizeCallback(window, framebuffer_size_callba
glfwSetMouseButtonCallback(window, mouse_button_callback);

// 初始化GLAD
if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress))
{
    std::cout << "Failed to initialize GLAD" << std::endl;
    return -1;
}

// 创建着色器程序
unsigned int shaderProgram = createShaderProgram();
if (!shaderProgram) return -1;
```

VAO/VBO 设置：

顶点数组对象（VAO）和顶点缓冲对象（VBO）用于存储所有像素的坐标数据，每次渲染时更新 VBO 并绘制所有像素点（GL_POINTS）。

```
// 设置像素绘制的VAO和VBO（顶点数组对象和顶点缓冲对象）
unsigned int VAO, VBO;
glGenVertexArrays(1, &VAO);//创建VAO，管理顶点属性
glGenBuffers(1, &VBO);//创建VBO，存储顶点数据
glBindVertexArray(VAO);//绑定VAO
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(flo
glEnableVertexAttribArray(0);

// 启用点大小设置
glUseProgram(shaderProgram);
glEnable(GL_PROGRAM_POINT_SIZE);
//设置点大小为5像素
glUniform1f(glGetUniformLocation(shaderProgram, "pointSize"),
```

3.交互逻辑（鼠标与键盘）

键盘控制：

按 1/2/3 键切换绘制模式（DDA 直线 / Bresenham 直线 / 圆）。

按 C 键清空画布（清除所有像素）。

按 ESC 键退出程序。

鼠标点击（核心交互）：

回调函数 mouse_button_callback 处理左键点击，将屏幕坐标转换为世界坐标（注意 y = SCR_HEIGHT - ypos，因为屏幕坐标 Y 轴向下，而程序中 Y 轴向上）。

```
// 鼠标点击回调
void mouse_button_callback(GLFWwindow* window, int button, int act
{
    if (button == GLFW_MOUSE_BUTTON_LEFT && action == GLFW_PRESS)
    {
        double xpos, ypos;
        glfwGetCursorPos(window, &xpos, &ypos);

        // 转换y坐标，使左上角为(0,0)
        float y = SCR_HEIGHT - (float)ypos;
        float x = (float)xpos;

        if (drawingMode == 1 || drawingMode == 2) {  // 直线模式
            linePoints.push_back(glm::vec2(x, y));
            drawPixel(x, y, glm::vec3(0.0f, 0.0f, 0.0f));  // 标记

            if (linePoints.size() == 2) {  // 已选择两个点，绘制直线
                glm::vec2 p1 = linePoints[0];
                glm::vec2 p2 = linePoints[1];

                if (drawingMode == 1) {  // DDA算法
                    DDA_line(p1.x, p1.y, p2.x, p2.y, glm::vec3(0.0
                } else {  // Bresenham算法
                    Bresenham_line((int)p1.x, (int)p1.y, (int)p2.x
                }

                linePoints.clear();
            }
        }
        else if (drawingMode == 3) {  // 圆形模式
            if (!hasCircleCenter) {  // 选择圆心
```

直线模式（1 或 2）：

第一次点击：记录起点，绘制一个黑色标记点。

第二次点击：记录终点，调用对应算法（DDA 或 Bresenham）计算直线上的所有像素，加入 pixels 容器。

圆模式（3）：

第一次点击：记录圆心，绘制黑色标记点。

第二次点击：计算与圆心的距离作为半径，调用中点画圆算法计算圆上的所有像素，加

入 pixels 容器。

4.核心算法实现（手动光栅化）

这部分是实验的核心，完全手动计算需要点亮的像素，模拟光栅化过程：

（1）DDA 直线算法：

原理：通过计算 x 和 y 方向的增量，逐步从起点 “走” 到终点，每步取最近的整数坐标作为像素位置。

```
// DDA画线算法
void DDA_line(float x1, float y1, float x2, float y2, glm::vec3 c
{
    //计算起点到终点的总增量
    float dx = x2 - x1;
    float dy = y2 - y1;

    // 确定步数：取x和y方向增量绝对值的最大值
    int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);

    // 计算每步的增量
    float xIncrement = dx / steps;
    float yIncrement = dy / steps;

    // 绘制直线
    float x = x1, y = y1;
    for (int i = 0; i <= steps; i++)
    {
        //对当前坐标四舍五入，取最近的整数像素位置
        drawPixel(round(x), round(y), color);
        x += xIncrement;
        y += yIncrement;
    }
}
```

（2）Bresenham 直线算法：

原理：用整数运算替代 DDA 的浮点运算，通过决策参数判断下一个像素的位置，效率更高。

中点是否在理想直线的 “上方” 或 “下方”，决定了下一个像素的选择：若 F(中点) > 0：中点在直线上方，说明下一个像素应向 y 方向修正（选 $(x_0 +1, y_0 +1)$）；若 F(中点) < 0：中点在直线下方，说明下一个像素应向 x 方向修正

err 误差大于零说明中点在直线上方，仅沿 x 方向移动；小于零说明中点在直线下方，需要沿 x,y 都移动

```cpp
// Bresenham画线算法
void Bresenham_line(int x1, int y1, int x2, int y2, glm::vec3 colo
{
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y1);

    // 确定方向
    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y1 < y2) ? 1 : -1;

    int err = dx - dy;
    int x = x1, y = y1;

    while (true)
    {
        drawPixel(x, y, color);

        // 到达终点
        if (x == x2 && y == y2) break;

        int e2 = 2 * err;

        // x方向步进
        if (e2 > -dy)
        {
            err -= dy;
            x += sx;
        }
        // y方向步进
        if (e2 < dx)
        {
            err += dx;
            y += sy;
        }
    }
```

（3）中点画圆算法：

原理：利用圆的 8 对称性（计算一个点可得到另外 7 个对称点），通过中点与圆的位置关系（决策参数）判断下一个像素。

从(0, radius)开始，初始化决策参数 d = 1 - radius。

每步 x+1，通过 d 判断 y 是否需要 - 1：

若 d < 0：中点在圆内，下一个像素 y 不变，更新 d。

若 d ≥ 0：中点在圆外，下一个像素 y-1，更新 d。

调用 plotCirclePoints 绘制 8 个对称点（如(x,y)、(-x,y)、(y,x)等），确保圆的完整性。

```cpp
void midpoint_circle(int centerX, int centerY, int radius, glm::ve
{
    int x = 0;
    int y = radius;
    int d = 1 - radius;  // 初始决策参数

    // 绘制初始点
    plotCirclePoints(centerX, centerY, x, y, color);

    // 计算并绘制其他点
    while (x < y)
    {
        x++;
        if (d < 0)// 中点在圆内：下一个像素为(x+1, y)
        {
            // 选择E点
            d += 2 * x + 1;
        }
        else// 中点在圆外或圆上：下一个像素为(x+1, y-1)
        {
            // 选择SE点
            d += 2 * (x - y) + 1;
            y--;
        }
        //绘制当前（x,y)的8个对称点
        plotCirclePoints(centerX, centerY, x, y, color);
    }
}

// 绘制圆的8个对称点
void plotCirclePoints(int centerX, int centerY, int x, int y, glm:
{
    drawPixel(centerX + x, centerY + y, color);
    drawPixel(centerX - x, centerY + y, color);
    drawPixel(centerX + x, centerY - y, color);
    drawPixel(centerX - x, centerY - y, color);
```

5.渲染循环

程序主循环不断执行以下操作：

处理输入（键盘切换模式、清空画布等）。

清空屏幕（白色背景）。

将 pixels 中的所有像素坐标数据更新到 VBO。

遍历 pixels，为每个像素设置颜色并绘制（glDrawArrays(GL_POINTS, ...)）。

交换缓冲区，刷新屏幕，响应事件。

```cpp
// 渲染循环
while (!glfwWindowShouldClose(window))
{
    // 处理输入
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    // 切换绘制模式，1 2 3键
    if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS) {
        drawingMode = 1;//DDA直线算法
        linePoints.clear();
        hasCircleCenter = false;
        std::cout << "当前模式：DDA直线算法" << std::endl;
    }
    if (glfwGetKey(window, GLFW_KEY_2) == GLFW_PRESS) {
        drawingMode = 2;
        linePoints.clear();
        hasCircleCenter = false;
        std::cout << "当前模式：Bresenham直线算法" << std::end
    }
    if (glfwGetKey(window, GLFW_KEY_3) == GLFW_PRESS) {
        drawingMode = 3;
        linePoints.clear();
        hasCircleCenter = false;
        std::cout << "当前模式：中点画圆算法" << std::endl;
    }
    //按C键清空画布
    if (glfwGetKey(window, GLFW_KEY_C) == GLFW_PRESS) {
        pixels.clear();
        linePoints.clear();
        hasCircleCenter = false;
```

实验二：多边形的扫描转换与填充

实验目的：

1. 理解扫描转换的基本原理，学习如何将多边形的几何数据转换为栅格图形。

2. 掌握多边形的边表创建、活动边表管理和扫描线处理的实现方法。

3. 通过实际编程，增强对图形学算法的理解与应用能力。

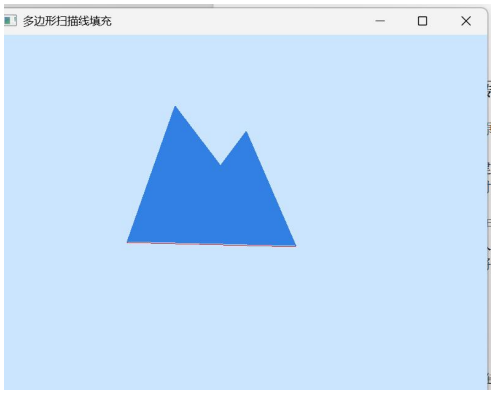
基本要求：

实现 x 扫描线填充（使用活性边表结构）

数据输入：允许用户输入多边形的顶点坐标，支持任意形状的简单多边形。

图形显示：利用图形库将转换后的多边形渲染到屏幕上。

运行结果：



实验内容：

1.扫描转换原理是将多边形的连续几何顶点（矢量图形）转换为屏幕上离散的像素点（栅格图形），核心是通过扫描线（水平直线）与多边形的交点确定填充范围。

```
// 活性边表(AEL)
std::vector<Edge> activeEdgeList;

// 对每条扫描线进行处理
for (int y = y_min; y <= y_max; y++) {
    // 将边表中当前y对应的边添加到活性边表
    auto it = edgeTable.find(y);
    if (it != edgeTable.end()) {
        //把边表中的边添加到活性边表中
        activeEdgeList.insert(activeEdgeList.end(), it->secon
        edgeTable.erase(it);//从边表中移除
    }



    // 移除活性边表中y_max <= 当前y的边，扫描线已经扫过了
    activeEdgeList.erase(
        std::remove_if(activeEdgeList.begin(), activeEdgeList
            [y](const Edge& e) { return e.y_max <= y; }),
        activeEdgeList.end()
    );

    // 更新活性边表中每条边的x坐标（沿扫描线移动）
    for (auto& e : activeEdgeList) {
        e.x += e.dx;//斜率倒数是y每增加1，x的变化量
    }

    // 按x坐标排序活性边表
    std::sort(activeEdgeList.begin(), activeEdgeList.end(), c
```

```
    // 更新活性边表中每条边的x坐标（沿扫描线移动）
    for (auto& e : activeEdgeList) {
        e.x += e.dx;//斜率倒数是y每增加1，x的变化量
    }

    // 按x坐标排序活性边表
    std::sort(activeEdgeList.begin(), activeEdgeList.end(), c

    // 成对取出交点并填充
    for (size_t i = 0; i < activeEdgeList.size(); i += 2) {
        if (i + 1 >= activeEdgeList.size()) break;//确保有成对

        int x_start = (int)std::round(activeEdgeList[i].x);//
        int x_end = (int)std::round(activeEdgeList[i + 1].x);

        // 填充从x_start到x_end的所有像素
        for (int x = x_start; x <= x_end; x++) {
            // 转换回标准化设备坐标
            float nx = (float)x / SCR_WIDTH * 2.0f - 1.0f;
            float ny = 1.0f - (float)y / SCR_HEIGHT * 2.0f;
            filledPixels.push_back(glm::vec2(nx, ny));
        }
    }


sFilled = true;
```

创建了一个活性边表，存储扫描线与多边形边的交点 x，这条边 y 的最大值，以及通过边的斜率的倒数计算出扫描线每增加 1，可以求出新的交点的 x 坐标。

填充时，将扫描线从下到上进行扫描，成对取出交点并在两点之间进行填充。

2. **同时支持用户交互输入多边形顶点**，通过鼠标回调函数（mouse_callback）实现用户输入：左键点击添加顶点，右键点击清除多边形。顶点坐标从屏幕坐标转换为标准化设备

坐标（适配 OpenGL 渲染），支持任意简单多边形（凸 / 凹均可，只要顶点按顺序输入）。

```
// 鼠标回调函数 - 用于输入多边形顶点
void mouse_callback(GLFWwindow* window, int button, int action, i
{
    //按下左键添加顶点
    if (button == GLFW_MOUSE_BUTTON_LEFT && action == GLFW_PRESS)
    {
        double xpos, ypos;
        glfwGetCursorPos(window, &xpos, &ypos);//获取鼠标坐标

        // 把屏幕坐标转换为标准化设备坐标
        float x = (float)xpos / SCR_WIDTH * 2.0f - 1.0f;
        float y = 1.0f - (float)ypos / SCR_HEIGHT * 2.0f;

        //将顶点添加到多边形顶点列表
        polygonVertices.push_back(glm::vec2(x, y));
        isFilled = false; // 新添加顶点后需要重新填充
    }
    else if (button == GLFW_MOUSE_BUTTON_RIGHT && action == GLFW_
    {
        // 右键清除多边形
        polygonVertices.clear();
        filledPixels.clear();
        isFilled = false;
    }
}
```

实验三：裁剪算法

实验目的：

1. 理解课堂上讲到的 Cohen-Sutherland 算法的核心思想。

2. 掌握 Liang-Barsky 裁剪算法的参数化表示，完成程序设计。

3. 体会参数表达和隐式表达在不同应用场合的优缺点。

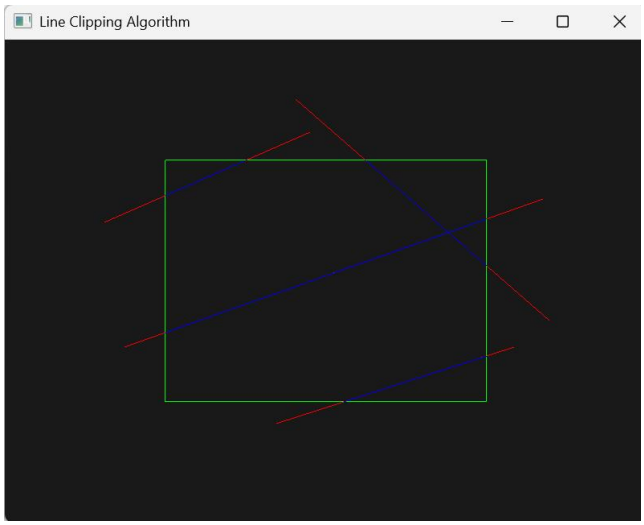
基本要求：

1. 用户交互画出窗口和待裁剪的线段（可重复给出，可移动、修改两端点），实时画出裁剪效果。

2. 画线的命令可以调用现成的 API（而不是逐像素地画）。

3. 窗口的“四把刀”分别进行裁剪，如果“注释掉三把刀”，只留“一把刀”，裁剪效果又会如何？

其他情况又会如何？请确保您的代码可以在验收时进行修改并展示效果。



实验内容：

（1）**Cohen-Sutherland 算法：**通过 “区域码” 标记线段端点与裁剪窗口的位置关系，利用位运算快速判断线段是否完全在窗口内（直接保留）、完全在窗口外（直接丢弃）或部分可见（需裁剪）。对部分可见线段，逐次与窗口边界（左、右、上、下）求交，用交点替

换外部端点，直到线段完全在窗口内。

```
// Cohen-Sutherland 裁剪算法
bool cohenSutherlandClip(Line& line) {
    Point p1 = line.start;//线段起点
    Point p2 = line.end;//线段终点

    int code1 = computeRegionCode(p1.x, p1.y);
    int code2 = computeRegionCode(p2.x, p2.y);

    while (true) {//循环直到线段完全在窗口内或完全在窗口外

        if (!(code1 | code2)) {
            // 线段完全在窗口内
            line.start = p1;
            line.end = p2;
            return true;
        } else if (code1 & code2) {
            // 线段完全在窗口外
            return false;
        } else {
            // 线段部分可见，需要裁剪
            //取在外部的端点的区域码（确定裁剪边界）
            int codeOut = code1 ? code1 : code2;
            Point p;//存储裁剪后的交点

            //根据区域码确定裁剪边界，计算线段与边界的交点
            if (codeOut & TOP) {
                p.x = p1.x + (p2.x - p1.x) * (clipTop - p1.y) / (p
                p.y = clipTop;
            } else if (codeOut & BOTTOM) {
                p.x = p1.x + (p2.x - p1.x) * (clipBottom - p1.y) /
```

（2）Liang-Barsky 算法

将线段用参数 u 表示（u∈[0,1]，u=0 为起点，u=1 为终点），通过计算线段与窗口四个边界的交点对应的 u 值，确定可见区间[u1, u2]。若 u1 <= u2，则线段在 u∈[u1, u2]的部分可见；否则完全不可见

```
// Liang-Barsky 裁剪算法
bool liangBarskyClip(Line& line) {
    Point p1 = line.start;
    Point p2 = line.end;

    float dx = p2.x - p1.x;//线段x方向增量
    float dy = p2.y - p1.y;//y方向增量

    float p[4] = { -dx, dx, -dy, dy };//左右下上四个边界
    //起点与边界的距离
    float q[4] = { p1.x - clipLeft, clipRight - p1.x, p1.y - clip

    float u1 = 0.0f, u2 = 1.0f;//线段参数u的可见区间

    //遍历四个边界
    for (int i = 0; i < 4; i++) {
        if (p[i] == 0) {
            // 线段平行于边界
            if (q[i] < 0) return false;//起点在边界外则线段完全不可
        } else {
            //不平行则计算参数u
            float r = q[i] / p[i];
            if (p[i] < 0) {//u1的值由线段从外向内遇到的矩形边界所决定
                u1 = std::max(u1, r);
            } else {//u2的值由线段从内向外遇到的边界的值决定
                u2 = std::min(u2, r);
            }
        }
    }
```

实验结论：

（1）隐式表达（Cohen-Sutherland）：

优点：通过区域码和边界比较判断位置关系，逻辑直观，易于理解和实现。

缺点：可能需要多次求交（最多 4 次），效率较低；对每条边界单独处理，代码冗余。

（2）参数化表达（Liang-Barsky）：

优点：用参数 u 一次性计算所有边界的交点，只需 1 次循环即可确定可见区间，效率更高；数学表达简洁，适合计算机高效计算。

缺点：参数化逻辑较抽象，理解难度稍大；对平行于边界的线段需要特殊处理（p[i]=0 的情况）。

实验四：Bézier 曲线与 B 样条

实验目的：

掌握 Bézier 曲线与 B 样条的原理与基本生成过程

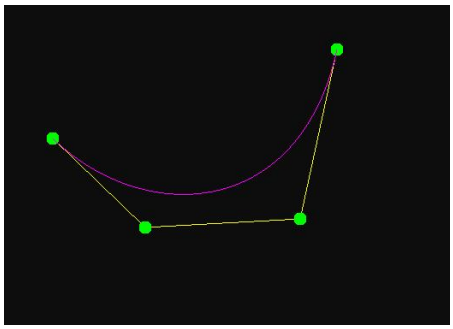
基本要求：

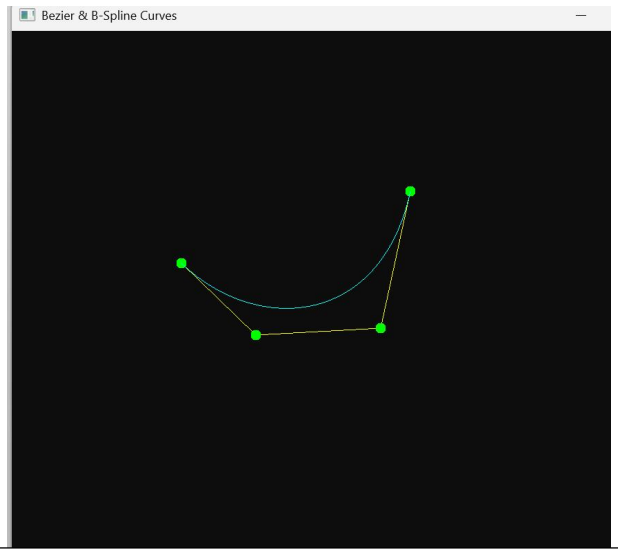
实现 de Casteljau 算法来绘制使用不同数量的控制点表示 Bézier 曲线

基于 de boor 割角算法来绘制使用不同数量的控制点表示 B 样条曲线

支持 insert/delete/move 控制点，同时画出控制顶点/控制多边形/样条曲线。

运行结果：







实验内容：

1. de Casteljau 算法

Bézier 曲线没有局部性，任意参数 t 对应的点都依赖全部 n+1 个控制点（n 为次数）。算法过程是：从全部 n+1 个控制点开始；每次迭代用固定系数(1-t)和 t 对相邻点插值，点数量逐层减 1；经过 n 次迭代后，仅剩的一个点就是曲线点。这种全局性导致 Bézier 曲线修改一个控制点会影响整个曲线

deCasteljau 函数：核心实现算法。通过迭代进行线性插值，第 k 层的点由第 k-1 层相邻两点插值得到，最终返回唯一的曲线点

```
// de Casteljau 算法计算Bézier曲线点
Point deCasteljau(const std::vector<Point>& points, float t) {
    std::vector<Point> temp = points;//复制控制点，避免修改原始数据
    int n = temp.size();//控制点数量

    for (int k = 1; k < n; k++) {
        // 计算第k层的每个点（第k层有n-k个点）
        for (int i = 0; i < n - k; i++) {
            // 计算第k层的每个点（第k层有n-k个点）
            //每个点是上一层相邻两点的线性插值
            temp[i].x = (1 - t) * temp[i].x + t * temp[i + 1].x;
            temp[i].y = (1 - t) * temp[i].y + t * temp[i + 1].y;
        }
    }
    //最终只剩下一个点，这个点就是曲线上对应t的位置
    return temp[0];
}
```

generateBezierCurve 函数：通过均匀采样 t∈[0,1]（共 bezierSegments+1 个点），调用 deCasteljau 计算每个点，生成完整曲线。

```
// 生成Bézier曲线
void generateBezierCurve() {
    bezierCurve.clear();//清空

    if (controlPoints.size() < 2) return;//至少两个控制点才能生成曲

    for (int i = 0; i <= bezierSegments; i++) {
        float t = static_cast<float>(i) / bezierSegments;//参数t
        Point p = deCasteljau(controlPoints, t);//曲线上的点
        bezierCurve.push_back(p);
    }
}
```

2. de boor 算法：“局部控制”，即参数 t 对应的曲线点仅由 t 所在节点区间（称为 “span”）关联的 k+1 个控制点决定（k 为次数）

De Boor 算法的 alpha 系数是动态的，由节点向量中 t 所在区间的长度和 t 的位置计算（alpha = (t - 左节点) / (右节点 - 左节点)），不同节点区间的系数计算方式不同；

deBoor 函数核心实现：找到 t 所在的节点区间[knots[span], knots[span+1])；从控制点中选取 span-k 到 span 的 k+1 个点作为初始点；递归插值 k 次，最终得到 t 对应的曲线点：

```
// 计算B样条基函数（决定了每个控制点对曲线形状的影响权重
//计算第 i 个 k 次 B 样条基函数在参数 t 处的值
float bsplineBasis(int i, int k, float t, const std::vector<float
    if (k == 0) {
        //0 次（k=0）B 样条基函数是最简单的 “矩形函数”：
        //当参数 t 落在节点向量的区间 [knots[i], knots[i+1]) 内时
        //基函数值为 1；否则为 0
        return (t >= knots[i] && t < knots[i + 1]) ? 1.0f : 0.0f;
    }

    float denom1 = knots[i + k] - knots[i];//分母
    float denom2 = knots[i + k + 1] - knots[i + 1];

    float term1 = 0.0f, term2 = 0.0f;
    //k 次基函数由两个 k-1 次基函数加权求和得到，
    //权重由参数 t 和节点向量的区间长度决定
    if (denom1 != 0) {
        term1 = (t - knots[i]) / denom1 * bsplineBasis(i, k - 1, t
    }

    if (denom2 != 0) {
        term2 = (knots[i + k + 1] - t) / denom2 * bsplineBasis(i +
    }

    return term1 + term2;
}
```

generateBSplineCurve 函数：均匀采样 t∈[0,1]，映射到有效节点区间，调用 deBoor 计算每个点，生成完整曲线。

```cpp
// 生成均匀节点向量knots:n为控制点数量，k为B样条次数
std::vector<float> generateUniformKnots(int n, int k) {
    std::vector<float> knots;
    int totalKnots = n + k + 1;

    // 前k+1个节点为0（起点重复k+1次，确保曲线从第一个控制点附近开始）
    for (int i = 0; i <= k; i++) {
        knots.push_back(0.0f);
    }

    // 中间均匀节点
    for (int i = 1; i <= n - k - 1; i++) {
        knots.push_back(static_cast<float>(i) / (n - k));
    }

    // 后k+1个节点为1
    for (int i = 0; i <= k; i++) {
        knots.push_back(1.0f);
    }

    return knots;
}
```

3. 对控制点的选中删除移动：

Insert（添加控制点）：左键点击空白区域时，mouse_button_callback 中 selectedPointIndex == -1，触发添加新控制点到 controlPoints，并调用 generateBezierCurve 和 generateBSplineCurve 更新曲线。

Delete（删除控制点）：右键点击控制点时，mouse_button_callback 中通过 findNearestControlPoint 找到最近的控制点索引，调用 controlPoints.erase 删除，并更新曲线。

Move（移动控制点）：左键点击选中控制点后拖拽，cursor_position_callback 中实时更新选中控制点的坐标（controlPoints[selectedPointIndex] = glPos），并更新曲线。

```
// 鼠标点击回调：处理控制点的添加、选择、删除
void mouse_button_callback(GLFWwindow* window, int button, int act
    double xpos, ypos;
    glfwGetCursorPos(window, &xpos, &ypos);//获得鼠标位置
    Point glPos = screenToOpenGL(xpos, ypos);

    if (button == GLFW_MOUSE_BUTTON_LEFT) {
        if (action == GLFW_PRESS) {//左键按下
            selectedPointIndex = findNearestControlPoint(glPos);
            if (selectedPointIndex != -1) {//找到控制点，标记为选中
                isDragging = true;
                controlPoints[selectedPointIndex].selected = true;
            } else {
                // 添加新控制点
                controlPoints.push_back(glPos);
                //生成新的曲线
                generateBezierCurve();
                generateBSplineCurve();
            }
        } else if (action == GLFW_RELEASE) {//左键释放
            if (isDragging && selectedPointIndex != -1) {
                controlPoints[selectedPointIndex].selected = false
            }
            isDragging = false;
            selectedPointIndex = -1;//消除选中索引
        }
```

实验五：网格操作

实验目的：

实现经典的网格简化算法（如 QEM）或曲面细分算法（如 Loop 细分）。理解网格简化和曲面细分的基本原理与实现过程。掌握网格模型文件的读取、处理与输出方法。能够控制网格简化和曲面细分的细节程度，并观察其对结果的影响。学习通过图形界面展示模型简化与细分的可视化效果。分析网格简化和曲面细分在计算机图形学中的实际应用与意义。

实验要求：

网格简化

使用 C++ 或其他熟悉的编程语言，选择一种经典的网格简化算法，编写程序读取包含大量三角形的网格模型文件，进行网格简化，并输出简化后的网格模型。实现简化过程中的算法细节，可控制简化后的网格细节程度。

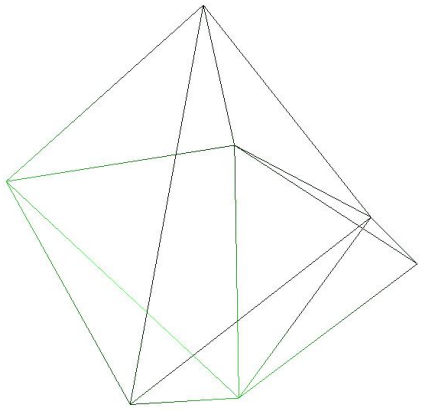
曲面细分

编写程序实现曲面细分，选择一种曲面细分算法（如 Loop 细分算法）。读取曲面模型文件，并输出细分后的曲面模型。控制曲面细分的细节程度，并处理细分后曲面的光滑度。

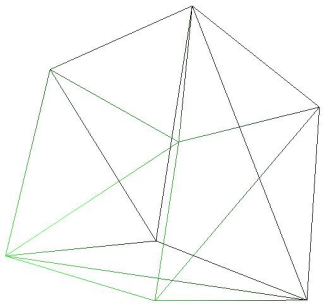
可视化

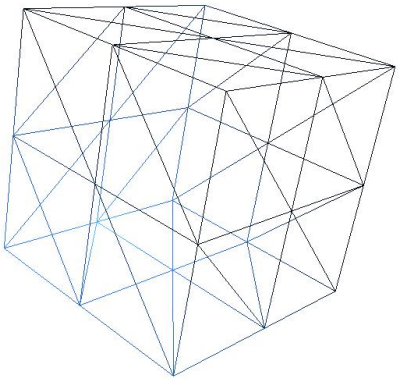
实现一个简单的图形界面，展示简化前后的网格模型和细分后的曲面模型的对比效果。

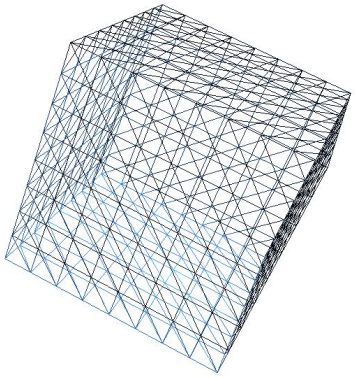
运行结果：



Mesh Operations - QEM Simplification & Loop Subdivision







实验内容：

（1）QEM 算法的核心目的是在简化 3D 网格（比如减少顶点和三角形数量）时，尽可能保留原模型的形状。它通过计算 “顶点合并误差” 来决定哪些顶点可以合并，而 Kp 正是计算这个误差的关键。

```
// 计算QEM算法中的误差矩阵
glm::mat4 computePlaneErrorMatrix(const glm::vec4& plane) {
    //plane四维向量（a,b,c,d)代表平面方程的参数
    glm::mat4 Kp;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            //kp是平面参数向量(a,b,c,d)的外积
            Kp[i][j] = plane[i] * plane[j];
        }
    }
    return Kp;
}
```

3D 网格中，每个顶点通常属于多个三角形面（比如立方体的一个顶点属于 3 个面），每个三角形面对应一个平面（即 plane 向量）。对于一个顶点 v，假设它属于 n 个平面，那么它的总误差矩阵 Q 是这 n 个平面的 Kp 之和：$Q = Kp_1 + Kp_2 + \ldots + Kp_n$ ，当我们想把两个顶点 v1 和 v2 合并成一个新顶点 v 时，合并的误差可以用 Q 计算：误差值 error = $v^T * Q_total * v$ 其中 Q_total 是 v1 和 v2 的总误差矩阵之和（Q1 + Q2），v 是 4 维齐次坐标（x,y,z,1）。

```
// 初始化顶点的误差矩阵：每个顶点的误差矩阵是其所属所有三角形平面的误差矩
void initializeVertexErrors(Mesh& mesh) {
    // 重置所有顶点的误差矩阵
    for (auto& v : mesh.vertices)
        v.K = glm::mat4(0.0f);

    // 遍历所有三角形，计算平面误差矩阵并累加到三个顶点
    for (const auto& t : mesh.triangles) {
        glm::vec3 p1 = mesh.vertices[t.v1].position;
        glm::vec3 p2 = mesh.vertices[t.v2].position;
        glm::vec3 p3 = mesh.vertices[t.v3].position;
        glm::vec4 plane = computePlane(p1, p2, p3);  // 三角形所在
        glm::mat4 Kp = computePlaneErrorMatrix(plane);  // 平面的误

        // 累加到三个顶点的误差矩阵
        mesh.vertices[t.v1].K += Kp;
        mesh.vertices[t.v2].K += Kp;
        mesh.vertices[t.v3].K += Kp;
    }
}
```

误差值越小，说明新顶点 v 越接近原顶点 v1 和 v2 所属的所有平面，合并后模型形状的失真越小。

为每条边计算合并后的最优顶点和误差；

每次选择误差最小的边进行合并（用新顶点替代 v1 和 v2，更新网格连接关系）直到网格简化到目标顶点数量。

```
// QEM,按误差从小到大合并顶点
void simplifyMeshQEM(const Mesh& input, Mesh& output, float ratio
    output = input;  // 复制原始网格
    output.initDeleted();  // 初始化删除标记

    if (output.triangles.size() < 2) return;  // 三角形太少无需简化

    // 步骤1：初始化顶点误差矩阵（每个顶点的误差是所属平面的误差和）
    initializeVertexErrors(output);

    // 步骤2：收集所有边并计算初始误差
    std::vector<Edge> edges = collectEdges(output);
    for (auto& e : edges) {
        computeEdgeError(output, e);  // 计算每条边的合并误差和最优
    }

    // 步骤3：用优先队列（最小堆）存储边，按误差从小到大排序
    std::priority_queue<Edge, std::vector<Edge>, std::greater<Edg

    // 步骤4：计算目标三角形数量（原始数量 * 简化比例，至少保留1个）
    int targetTriangles = std::max(1, (int)(output.triangles.size
    int currentTriangles = output.triangles.size();

    // 步骤5：迭代合并边，直到达到目标三角形数量
    while (currentTriangles > targetTriangles && !pq.empty()) {
        Edge bestEdge = pq.top();  // 取出误差最小的边
        pq.pop();

        int v1 = bestEdge.v1;
        int v2 = bestEdge.v2;
```

（2）Loop 细分是针对三角形网格的曲面细分算法，通过增加三角形数量使曲面更光滑，核心步骤：

为每条边生成一个新顶点（新顶点位置由相邻顶点加权平均计算）；

更新原始顶点位置（由自身和相邻顶点加权平均计算）；

将每个原始三角形分割为 4 个新三角形（原始顶点 +3 条边的新顶点）；

重复上述步骤，每级细分使三角形数量变为 4 倍，曲面更光滑。

```
// Loop细分算法，增加三角形数量
void subdivideMeshLoop(const Mesh& input, Mesh& output, int levels
    if (input.triangles.empty() || levels <= 0) {
        output = input;
        return;
    }

    Mesh current = input;
    // 按细分级别迭代（每级细分使三角形数量变为4倍）
    for (int level = 0; level < levels; level++) {
        Mesh next;// 下一级细分后的网格

        std::map<std::pair<int, int>, int> edgeVertices;

        // 复制原始顶点
        next.vertices = current.vertices;

        // 为每条边创建新顶点（Loop细分规则：每条边生成一个新顶点）
        for (const auto& triangle : current.triangles) {
            int edges[3][2] = {
                {triangle.v1, triangle.v2},
                {triangle.v2, triangle.v3},
                {triangle.v3, triangle.v1}
            };
            // 处理每条边
            for (int i = 0; i < 3; i++) {
                int v1 = edges[i][0];
                int v2 = edges[i][1];
                // 确保边的索引对有序（v1 < v2），
                //避免重复处理同一条边（如v1-v2和v2-v1视为同一条）
```

```
            }
        }

        // 分割每个三角形为4个新三角形（原始顶点+3条边的新顶点）
        for (const auto& triangle : current.triangles) {
            int v1 = triangle.v1;
            int v2 = triangle.v2;
            int v3 = triangle.v3;
            // 获取三条边对应的新顶点索引
            int e1 = edgeVertices[{std::min(v1, v2), std::max(v1,
            int e2 = edgeVertices[{std::min(v2, v3), std::max(v2,
            int e3 = edgeVertices[{std::min(v3, v1), std::max(v3,

            // 创建4个新三角形
            next.triangles.push_back(Triangle(v1, e1, e3));
            next.triangles.push_back(Triangle(v2, e2, e1));
            next.triangles.push_back(Triangle(v3, e3, e2));
            next.triangles.push_back(Triangle(e1, e2, e3));
        }

        current = next;
    }

    output = current;
    output.calculateNormals();

    std::cout << "Subdivided mesh: " << output.triangles.size() <<
```

结论：

通过 0-5 的实验，主要学习了

1.光栅化是将几何图形转化为屏幕像素的核心过程，实现两种经典算法：

DDA 画线算法：基于直线的斜率计算，通过增量方式逐点确定像素位置。例如，当斜率绝对值小于 1 时，x 轴每次递增 1，y 轴按斜率增量更新，避免浮点数乘法，仅通过加法实现连续像素生成，本质是用离散像素逼近连续直线。

Bresenham 画线算法：进一步优化 DDA 的浮点数运算，引入 “误差变量” 判断下一个像素位置。通过整数运算动态调整误差，当误差超过阈值时沿 y 轴步进，确保像素点始终贴近理想直线，效率高于 DDA。

画圆算法（如中点圆算法）：利用圆的对称性（仅计算 1/8 圆即可生成全圆），通过 “中点偏差” 判断像素位置。定义决策参数 d，根据中点与理想圆的位置关系（内 / 外）调整下一个像素，避免三角函数计算，用整数运算快速生成圆形像素集。

2.多边形扫描转换与填充：

核心是将多边形的顶点数据转化为屏幕上的填充像素，采用 x 扫描线填充算法，关键在于 “边表” 与 “活动边表” 的管理：

边表创建：遍历多边形所有边，记录每条边的起点、终点、斜率倒数（$\Delta x/\Delta y$）及下一个交点的 x 坐标，按边的最低 y 坐标分类存储，确保扫描线从下到上处理时能快速索引到相关边。

活性边表管理：扫描线逐行移动时，将当前 y 坐标对应的边加入活动边表，移除超出 y 范围的边，并对活动边表中的边按 x 坐标排序。

扫描线处理：对每条扫描线，遍历排序后的活动边表，将相邻边的 x 坐标区间内的像素填充为目标颜色，实现多边形的完整填充，本质是通过 “逐行扫描 + 区间填充” 将多边形转化为连续像素块。

3.线段裁剪算法

裁剪是保留图形在 “裁剪窗口” 内部分、丢弃外部部分的过程，实现两种经典算法：

Cohen-Sutherland 算法：通过 “区域码” 判断线段与窗口的位置关系。将窗口分为 9 个区域，每个点用 4 位二进制区域码（左、右、下、上）标识，通过区域码的逻辑运算（如 “与运算非零则线段全在窗外”“或运算为零则线段全在窗内”）快速筛选线段；对部分在窗内的线段，计算其与窗口边界的交点，逐步裁剪外部部分。

Liang-Barsky 算法：基于参数化直线表示（将线段表示为 $P(t)=P1+t*(P2-P1)$，$t\in[0,1]$），通过 “不等式约束” 计算 t 的有效区间。对窗口的 4 条边界（左、右、下、上），分别求解 t 的范围，最终取所有范围的交集，若交集非空则根据 t 值计算裁剪后的线段端点，参数化表示使其更易扩展到三维裁剪。

4.曲线生成算法：聚焦于参数曲线的生成，实现两种经典曲线的构造：

Bézier 曲线（de Casteljau 算法）：通过 “分层线性插值” 生成曲线。给定 n+1 个控制点，从控制点开始，每一层对相邻点按参数 $t\in[0,1]$进行线性插值，经过 n 层插值后得到唯一的点，即为曲线上对应 t 的位置；t 从 0 到 1 遍历，可生成完整的 Bézier 曲线，曲线始终在控制点构成的“控制多边形”内，且仅依赖端点和相邻控制点，便于交互调整。

B 样条曲线（de Boor 割角算法）：解决 Bézier 曲线 “全局控制” 的缺

陷（修改一个控制点会影响整条曲线），引入 “基函数” 和 “节点向量”。通过 de Boor 算法，按节点向量的分段区间，对局部控制点进行分层插值，生成的曲线具有 “局部性”（修改一个控制点仅影响相邻区间的曲线），且支持任意数量的控制点，可灵活调整曲线形状；同时支持控制点的增、删、移，实时更新控制多边形与曲线。

5.网格操作算法：针对 3D 网格模型的优化与细化，实现两种核心算法：

网格简化（QEM 算法）：核心是 “最小化形状失真”，通过 “二次误差度量” 评估顶点合并的代价。首先为每个顶点计算误差矩阵（由顶点所属三角形平面的误差矩阵累加得到），再为每条边计算 “合并误差”（合并后新顶点到所有平面的总距离平方）和 “最优顶点位置”（使误差最小的点）；用优先队列按误差从小到大合并边，直到达到目标三角形数量，最后清理无效顶点与三角形，本质是在减少模型复杂度的同时保留关键形状特征。

曲面细分（Loop 细分算法）：通过 “分裂三角形” 提升模型细节。每级细分中，先为每条边创建新顶点（避免重复处理共享边），再将每个原始三角形分裂为 4 个小三角形；标准 Loop 算法会通过顶点 “度”（共享该顶点的三角形数量）加权计算新顶点位置，确保细分后曲面平滑，避免褶皱；多次细分可将低精度模型（如立方体）转化为高精度平滑模型（如近似球体）。

通过实验，理解图形学的核心逻辑：从 **“几何定义” 到 “屏幕像素”** 的全流程，本质是 “离散化” 与 “逼近”—— 无论是直线的像素生成、圆的对称性利用，还是多边形的扫描填充、网格的细分与简化，都是用离散的计算（整数运算、增量更新、分层插值）逼近连续的几何图形，解决 “连续几何” 与 “离散屏幕” 的矛盾。

体会**交互与可视化**的实现逻辑：图形学不仅是算法设计，还需结合用户交互与实时渲染。例如，鼠标事件回调与矩阵变换的结合实现图形拖动，控制点的增删改实时更新曲线，网格简化前后的对比可视化，这些过程让我理解了 “算法逻辑” 如何通过 “编程实现” 转化为 “用户可感知的图形效果”。

认识不同算法的**适用场景**：例如，Cohen-Sutherland 算法适合简单窗口的快速裁剪，Liang-Barsky 算法更适合参数化场景；Bézier 曲线适合简单形状设计（如字体、logo），B 样条曲线适合复杂曲面（如工业设计）；QEM 算法用于模型轻量化（如游戏场景优化），Loop 细分用于模型高精度渲染（如影视特效），这些认知为后续复杂图形学应用奠定基础。